

ONTASK - CONNECTING PEOPLE WITH OPPORTUNITIES

PROJECT PHASE – II REPORT

Submitted by

DHRUVRAJSINH

CHAUHAN

SHAUMIL MISTRY

in partial fulfillment for the award of the degree of

**BACHELOR OF TECHNOLOGY In COMPUTER
SCIENCE AND ENGINEERING**

**Unitedworld Institute of Technology
Karnavati University**

Uvarsad, Gandhinagar, Gujarat-382422 November, 2025

KARNAVATI UNIVERSITY
Uvarsad, Gandhinagar, Gujarat-382422
Unitedworld Institute of Technology
BONAFIDE CERTIFICATE

Certified that this project report “**OnTask Connecting People with Opportunities**” is the Bonafide work of “**DHRUVRAJSINH CHAUHAN, SHAUMIL MISTRY**” who carried out the Project Work (Phase-II) / Project Work under my / our supervision.

Prof. Jatin Ambasana

SUPERVISOR

Assistant. Professor

Unitedworld Institute of Technology

Karnavati University

Dr. Puneet Sharma

HEAD OF THE INSTITUTION

Associate Dean

Unitedworld Institute of Technology

Karnavati University

Submitted for the project work viva-voce examination held on_____

INTERNAL EXAMINER

Signature:_____

EXTERNAL EXAMINER

Signature:_____

DECLARATION

I declare that this project report titled “**OnTask- Connecting People with Opportunities** ” submitted in partial fulfilment of the degree of B. Tech in (Computer Science Engineering/Artificial Intelligence and Machine Learning) is a record of original work carried out by me under the supervision of **Prof. Jatin Amabasana**, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Dhruvrajsinh Chauhan (20220702076) _____

Shaumil Mistry (20220702126) _____

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to **Karnavati University** for providing the facilities and academic environment necessary for the successful completion of our project. We extend our heartfelt thanks to our project guide, **Professor Jatin Amabasana**, for their constant guidance, valuable suggestions, and encouragement throughout the duration of this project. Their expertise and supervision were instrumental in the successful execution of this work. We are thankful to all faculty members who have directly or indirectly contributed to the development of this project. Their constructive feedback and motivation have greatly enhanced the quality of our work. Lastly, we wish to acknowledge the cooperation and collective effort of our team members, whose dedication and coordination have been vital in bringing this project to fruition.

Project Team

Dhruvrajsinh Chauhan

Shaumil Mistry

ABSTRACT

The current state of employment has drastically altered. Instead of the previously predominant centralized employment models of the past, there is now a movement away from these formats toward a more flexible decentralized model for labor. OnTask is one such platform and offers a complete, fully integrated labor supply chain providing everything an individual and/or organization need to connect individuals with the correct skills, manage projects and processes, and receive payment to their bank accounts via the MERN (MongoDB/Express.js/React.js/Node.js) stack. To help facilitate these functions, the OnTask digital labor platform has built a sophisticated multi-role authentication system that weaves together the "Employer" Providers and "Worker" Taskers with individual data-driven dashboards. OnTask has a unique technological differentiator: it offers an AI-based talent screening system for recruiters to efficiently employ rigorous and quantifiable selection criteria through the use of custom screening questions and automated performance measurement devices. OnTask employs advanced protocols that provide real-time communication (Socket.io) between users of the system. The core of the platform when it comes to the security architecture is the proprietary and secure Escrow Payment Gateway that is integrated with the Razorpay API. This transaction processing system utilizes "safe capture" and "safe release" logic.

TABLE OF CONTENTS

CHAPTER/SECTION	TITLE	PAGE NO.
	BONAFIDE CERTIFICATE	ii
	DECLARATION	iii
	ACKNOWLEDGEMENT	iv
	ABSTRACT	v
	TABLE OF CONTENTS	vi
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
CHAPTER 1	INTRODUCTION	1
1.1	General Overview	1
1.2	Background and Motivation	1
1.3	Problem Statement	2
1.4	Objectives of Phase 2	2
1.5	Scope of the Work	2
CHAPTER 2	LITERATURE REVIEW	4
2.1	Study of Existing Systems	4
2.2	Comparative Analysis	4
2.3	Gaps In Existing Platform	5
2.4	OnTask Proposed Solution	5
CHAPTER 3	SYSTEM ANALYSIS AND DESIGN	7
3.1	Requirements	7
3.2	Feasibility Study	7
3.3	Use Case Analysis	8
3.4	System Architecture	9
3.5	Database Design	9
3.6	Architectural Patterns	9
3.7	Interface Design	10
CHAPTER 4	IMPLEMENTATION	11
4.1	System Architecture	11

4.2	Project Organization	11
4.3	Database Implementation	11
4.4	Backend Implementation Details	12
4.5	Frontend Implementation Details	13
CHAPTER 5	TESTING AND EVALUATION	14
5.1	General	14
5.2	Testing Tiers	14
5.3	Test Case Tables	14
5.4	Performance Benchmarks	16
CHAPTER 6	CONCLUSION AND FUTURE WORK	29
6.1	Conclusion	29
6.2	Limitations	29
6.3	Future Enhancements	29
	APPENDICES	31
	REFERENCES	38

TABLE OF TABLES

TABLE	TITLE	PAGE NO.
2.2.1	Features Comparison Table	4
5.3.1	Authentication and Authorization Test Cases	14
5.3.2	Job and Recruitment Module Test Case	15
5.3.3	Escrow and Financial Security Test Case	15
5.4.1	Performance Evaluation	16

TABLE OF FIGURES

TABLE	TITLE	PAGE NO.
3.2.1	System Workflow	8
5.5.1	Signup Page	16
5.5.2	Job Seeker Signup Step-1	17
5.5.3	Job Seeker Signup Step-2	17
5.5.4	Job Seeker Home Page	18
5.5.5	Job Seeker Finding Jobs	18
5.5.6	Job Seeker Applied Jobs	19
5.5.7	Job Seeker Active Contracts	19
5.5.8	Job Seeker Completed Jobs	20
5.5.9	Job Seeker Submit Updates	20
5.5.10	Job Seeker Earnings Window	21
5.5.11	Job Seeker Message Section	21
5.5.12	Job Seeker Notification Section	22
5.5.13	Job Seeker Profile Page	22
5.5.14	Job Seeker Profile Settings	23
5.5.15	Job Provider Signup Step-1	23
5.5.16	Job Provider Signup Step-2	24
5.5.17	Job Provider Signup Step-3	24
5.5.18	Job Provider Signup Step-4	25
5.5.19	Job Provider Home Page	25
5.5.20	Job Provider Job Post	26
5.5.21	Job Provider Posted Jobs	26
5.5.22	Applications of Job Seekers	27
5.5.23	Job Provider Approve Progress	27
5.5.24	Job Provider Active Jobs	28
5.5.25	Job Provider Find Job Seekers	28
7.1.1	MVC Architecture	31
7.1.2	Push Notification Config	32

7.1.3	Send OTP to Mail	32
7.1.4	Auth Controller	33
7.1.5	Payment Controller	33
7.1.6	Application Controller	34
7.1.7	Wallet Controller	34
7.1.8	Earning React Page	35
7.1.9	Message React Page	35
7.1.10	Reset Password React Page	36
7.1.11	Feedback Model React UI	36
7.1.12	AI Chatbot React UI	37
7.1.13	Tasker Search React Page	37

CHAPTER 1

INTRODUCTION

1.1 General Overview

OnTask is a unique and ground-breaking decentralized labor management platform created to empower today's workforce. As "work-from-anywhere" becomes the norm in this age of modern technology, On Task serves as a digital bridge connecting local skilled labour with their global and local employers. Unlike most job boards, On Task is a transaction-centric recruiting platform; all hiring activity represents commitment on behalf of both sides, creating a high-integrity marketplace for both job seekers (Taskers) and job employers (Providers); where both parties can work together transparently.

Based on the belief that recruiting is not simply a matter of matching, but rather an entire process of recruiting from the initial posting through to the final payment, On Task incorporates identity verification, requirement definition, screening/candidate selection, real time negotiation, milestone tracking and secure payment in one simple, web-based application.

1.2 Background and Motivation

OnTask was established to help solve the inefficiencies caused by unregulated/informal work arrangements in the "gig economy", especially for smaller/simpler types of projects, such as small, short-duration coding tasks; casual graphic design work; and other local service-based gigs. Generally speaking, such types of projects lack a formal system for governance, resulting in several significant problems for both workers as well as companies and users of those types of jobs.

- The "Voucher" Problem: Many workers are paid after completing the work ("post-delivery"), creating a potentially high level of risk that they may not get paid.
- The "Screening" Problem: There is insufficient time available for small-scale employers to filter through potentially hundreds of applicants for just one position.
- The "Context" Problem: The use of platforms such as WhatsApp to discuss job requirements and then having to track the status of each job using/within Excel results in a loss of data.

As a result, OnTask created an Organized Digital Governance Layer for managing gig economy jobs, which provides an official framework for organizing all types of

informal work.

1.3 Problem Statement

This project, therefore, addresses the fragmented recruitment-to-payment process for gig economy companies and their workers.

Specifically:

- **Trust Deficit:** How does a Tasker trust a new Provider to actually pay them? How does a Provider trust a Tasker to complete the task they are assigned?
- **Siloed Communication:** There is no connection between where you recruit potential workers (LinkedIn) and where you pay your workers (PayPal, GPay).
- **Verification Gap:** Currently, there is no ability for a Provider to submit and verify evidence of completion of work prior to issuing a payment to
- **Recruiter Fatigue:** There is the difficulty and frustration of screening through anywhere from 50–100 candidates for one short-term hire.

1.4 Objectives of Phase 2

OnTask is designed to accomplish the following core goals:

- Use of a unified interface that provides a single solution for all your needs related to registration; hiring; communications; tracking and monitoring at all levels of your project.
- Screening with enhanced scoring system; utilizing checklist type questions during screening that provide recruiters with an avenue to score candidates adequately and produce a numerical score representing the candidate's fitness for the position.
- Integration of escrow financial transactions; utilizing an escrow service (Razorpay) and internal wallet to facilitate the “safe capture” or “safe release” of funds.
- Use of real-time communication using web-sockets and notification updates for rapid delivery of information and a seamless user experience.
- Ability to provide transparency of progress made by workers using documented evidence (images) and percentage completion of tasks and verification by providers prior to disbursement of payment.

1.5 Scope of the Work

OnTask project has an extensive scope with the scope of work encompassing the

complete engagement lifecycle:

- Registration / Profiling: Rich profile builders of the two classes (Tasker and Provider) like skill tagging and portfolio management.
- Job Lifecycle Management: Provider tools to create, control and monitor jobs with a particular budget and requirement header.
- Advanced Recruitment: Multi-parameter search algorithms, application tracking system (ATS), built in interview/screening logic.
- Escrow-Based Finance: A more powerful Project Wallet system dealing with deposits, escrow hold and final payouts including automatic calculation of tax/fees.
- Instant Communication: A bi-directional, persistent messaging engine that retains all the conversations pertaining to their projects in the context of the job.
- Evidence-Based Delivery: A module through which workers would post status updates and supporting documents/images, which would offer a verifiable audit trail of work completed.

CHAPTER 2

LITERATURE REVIEW

2.1 Study of Existing Systems

In order to create state of the art platform, it was necessary to study the industry giants that already control the market:

2.1.1 *LinkedIn*

Linked In is the new king of professional networking and long-term recruiting. Nevertheless, it is mainly interested in the area of connections and talent discovery. It does not have built-in project management and escrow-based payment loops of short-term hourly or fixed-price gigs, which is inappropriate to the fast-paced hire-to-pay cycle of the gig economy.

2.1.2 *Upwork & Freelancer.com*

These are the nearest competitors of OnTask. Their services include escrow and global recruiting. Their complexity and high fee structure tends to put off small-scale providers and local markets, however. Moreover, they have communication tools that are frequently subservient to their bidding engines and that may result in race-to-the-bottom pricing that is detrimental to quality.

2.1.3 *Fiverr*

Fiverr has transformed the service as a product (Gig) model. Although it is very useful in the case of standardized services (e.g., "I will design a logo"), it is poor when dealing with complex, custom projects that need extensive screening and tracking of progress.

2.2 Comparative Analysis

Table 2.2.1: Features Comparison Table

Feature	LinkedIn	Upwork	Fiverr	OnTask
Primary Focus	Networking	Bidding/Escrow	Service Catalog	Recruitment-to-Escrow
Payment Security	None (Third-party)	High (Escrow)	High (Prepaid)	Maximum (Integrated Wallet)
Screening Tools	Resume-based	Skills/Tests	Rating-based	Custom Screening Questions
Real-time Comms	Basic Chat	Integrated	Basic Chat	Persistent Socket.io Engine
Progress Support	None	Time-tracking	Delivery-only	Evidence-upload/Verification

2.3 Gaps In Existing Platforms

In the study, one gap was established as a critical gap of the Grand Canyon: the junction of Recruitment and Real-time Payment.

- The Hired Wall: Most of the platforms consider Hired to be a terminal. As a matter of fact, it is only the start of the most dangerous stage of the project.
- Payment Fragmentation: Transferring payment through a platform into a bank is a Break in the Trust Chain. In case of a dispute, there is no centralized work to payment record.
- Context Loss: In individual app (Slack, WhatsApp) messaging, the requirements have no single point of truth.

2.4 OnTask Proposed Solution

2.4.1 Overview

The solution that has been proposed is called OnTask. This is a complete online platform that connects employers with talent (taskers) who have the skills to do the tasks requested by employers. OnTask is different than ordinary job boards in that it provides job seekers and employers with real-time communication capabilities, an escrow-based (secure) method of paying for completed work, and a real-time notification system to promote transparency and demonstrate reliability when working through OnTask.

OnTask provides an efficient and effective means of managing every aspect of a task's lifecycle, from posting/bidding for a task to collaborating on the task in real time (and Finalizing payment for the task after the task has been completed by the tasker).

2.4.2 Architecture

OnTask employs the Client-Server architecture to create a decentralized way of keeping track of the state of the application and communicating with users in real time.

2.4.2.1 The Component Diagram of OnTask

The OnTask system is made up of 3 major layers.

1. Presentation Layer (Frontend) - The presentation layer of OnTask is a single page application built using React.js and Vite. The Presentation Layer makes requests to the Backend via RESTful API's and maintains a persistent WebSocket connection for real-time capabilities.

2. Application Logic Layer (Backend) - The Backend of OnTask is developed on a Node.js and Express.js server that will handle the Authentication of users, Business Logic functions, the Uploading Files and sending data from the server via Socket Events.

3. Data Layer (Database) - In OnTask, the MongoDB (NoSQL) database is used as the Data Layer. It stores unstructured data such as users' profiles, job descriptions and messages. This Data Layer provides high availability and scalability as well.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Requirements

3.1.1 Functional Requirements (FRs)

- User Management: Registration, Logging In and Out, Editing Profiles, and Storing/Updating Banking Details.
- Job Management: Creating New Jobs, Editing Jobs, Searching for Jobs (Filtering by Categories / Salaries / Locations).
- Application Management: Submitting Applications with Screening Questions, Withdrawing Applications, and Reviewing Applications.
- Messaging System: Provides room-based messaging between Provider and Tasker upon Hiring or Shortlisting.
- Wallet System: Adding Funds (Razorpay), Deducting from Escrow, Releasing/Withdrawing Payments.
- Evidence Reporting: Taskers will have the ability to upload photos of completed work as proof of completion, known as "Progress Proof".

3.1.2 Non-Functional Requirements (NFRs)

- Integrity: Financial data must never "leak" or "vanish" (Atomic MongoDB transactions for Wallet updates only).
- Speed: The system must respond to all User Interface actions within 200 ms (i.e., buttons/tabs) and under 1 second for all back-end API calls.
- Scalability: The system must be able to support 1,000 or more concurrent WebSocket connections without degradation of performance on the server.
- Security: All communications must be conducted using HTTPS Protocol, Application Inputs must be sanitized against XSS attacks, and Passwords must be stored as hashed values using Bcrypt algorithm.

3.2 Feasibility Study

- Technical Feasibility - The use of JavaScript for the entire application stack reduced the possibility of errors caused by context switching when the development team deployed the application (via Cloud Providers: Vercel/Render) into production and increased the possibility of scalability.
- Economic Feasibility - This solution will replace third-party platforms, which typically charge high transaction fees, with a self-managed platform to make this project economically feasible for small business owners.

- Operational Feasibility - The UI makes use of "Material Design" guidelines to facilitate use of the application by users who may be not be technology savvy to administer their hiring process.

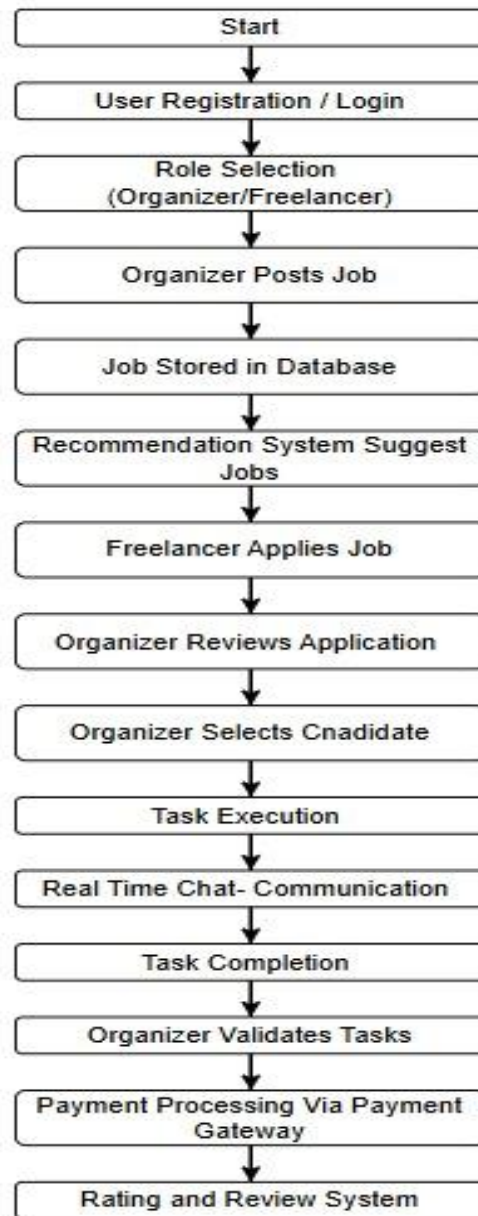


Figure 3.2.1: System Workflow

3.3 Use Case Analysis

- Provider Use Cases: Deposit Project Budget, Candidate Filter by Score, Verify Work Evidence, Release Payment.

- Tasker Use Cases: Job Search for High Budget, Real Time Query, Progress Updates w/Photo, Request Bank Withdrawals.

3.4 System Architecture

OnTask utilizes a Client-Server Architecture with a persistent WebSocket layer.

- Frontend: React.js SPA (Single Page Application) using Hooks for state management and Context for Auth/Socket propagation.
- Backend: Node.js/Express.js REST API.
- Real-time Layer: Socket.io for event-driven updates.
- Database: MongoDB Atlas (Cloud NoSQL).
- Payments: Razorpay Integrated SDK.

3.5 Database Design (Entity Relationship)

The database is designed for High Read Performance:

- User Document:
``id`, `name`, `email`, `role` (Tasker/Provider)`
``walletBalance`, `escrowBalance``
``portfolio` (Skills, Experience)`
``bankDetails` (Encrypted)`
- Job Document:
``title`, `description`, `budget`, `status` (Open/Funded/Completed)`
``screeningQuestions` [Array of Strings]`
``hiredCount`, `requiredWorkers``
- Application Document:
``jobId` (Ref), `userId` (Ref)`
``answers` [Array of Strings]`
``status` (Pending/Shortlisted/Hired)`
- Message Document:
``senderId`, `receiverId`, `content`, `timestamp``

3.6 Architectural Patterns

- Middleware Pattern: Used for Auth verification (isLoggedIn, isProvider) on the

backend.

- Observer Pattern: Used in the frontend to listen for WebSocket events and update the "Notification Bell" and "Chat Bubble" in real-time.
- Strategy Pattern: Used for payment processing to allow switching between payment gateways in the future.

3.7 Interface Design

The User Experience (UX) is built on the "Frictionless Hiring" philosophy.

- The Three-Click Rule: A user should be able to reach any critical action (Apply, Hire, Reject, Pay) in three clicks from the dashboard.
- Visual Hierarchy: Primary buttons (Apply/Hire/Reject) use vibrant colors to guide the user's focus.
- Real-time Indicators: Green dots for online users and red number badges for unread messages ensure users never miss an update.

CHAPTER 4 IMPLEMENTATION

4.1 System Architecture

During implementation, OnTask focused on building its architecture into software that was both usable and secure while also supporting scale. An Agile methodology was employed during the development process, allowing relatively comparable improvements to both backend API and frontend UI in all iterations.

4.2 Project Organization

The project is organized into two principal directories, frontend and backend.

4.2.1 Backend Directory (/backend)

- `models/`: Mongoose schemas for data.
- `controllers/`: Core business logic and handling requests.
- `routes/`: where API endpoints definitions are mapped into controllers.
- `config/`: Passport configuration. Database connections and environment variables in node.
- `server.js`: the starting point where Express server and Socket. io are initialized.

4.2.2 Frontend Directory (/frontend)

- `src/components/`: Reusable UI components (Headers, Buttons, Cards).
- `src/pages/`: The main views for the application (Dashboards, Login, Job Listing).
- `src/context/`: For Authentication and Sockets global state management.
- `src/utils/`: Helper functions for making API calls and formatting data.

4.3 Database Implementation

MongoDB database with 5 collections Mongoose models represent each collection.

4.3.1 User Schema (User.js)

Houses user profiles, Roles, Auth Ids (Google/Local), and verification status. It utilizes Bcrypt to hash passwords.

```
const userSchema = new mongoose.Schema({  
  name: {type:String, required:true},
```

The email field will be of type string but marked as unique so that two users don't have the same username (required, making sure no MSSQL or nonexistent in NoSQL).

```
password: { type: String },
role: {type:String,enum:['provider','freelancer'],default:'freelancer'},
isVerified: { type: Boolean, required:false }
walletBalance: { type : Number, default : 0 }
}, { timestamps: true });
```

4.3.2 Job Schema (Job.js)

Documents job description, the budget of it and its length as well as relationship between poster and applying character.

```
const jobSchema = new mongoose. Schema({
title: { type:String, required:true}}
description: { required, type: String },
budget: { type: Number, required : true },
postedBy: { type: mongoose. Schema. Types. ObjectId, ref: 'User' },
status: { type: String, enum : ['pending', 'open', 'in-progress','completed'], default:
open }
}, { timestamps: true });
```

4.4 Backend Implementation Details

4.4.1 Real-Time Messaging using Socket.io

In the server.js file, a persistent connection pool is created using a Map object that associates each user with their corresponding socket. This allows the server to easily send messages directly to specific users.

```
io.on('connection', (socket) => {
socket.on('registerUser', (userId) => {
    userSockets.set(userId, socket.id);
});
// Further develop message routing code.
});
```

4.4.2 RESTful API and Controllers

To manage the various business processes that are being performed, all Controllers act as business process controllers. For example, the jobController.js file controls filtering and pagination within the Job Marketplace, whereas the authController.js file deals with session tokens and so on.

4.5 Frontend Implementation Details

To maintain a global state of the authenticated user and maintain an active socket connection across the entire app, the Frontend employs Context API.

```
export const AuthProvider = ({ children }) => {  
  const [user, setUser] = useState(null);  
  const [socket, setSocket] = useState(null);  
  // Business logic for obtaining user profile and initializing socket  
};
```

CHAPTER 5

TESTING AND EVALUATION

5.1 General

5.1.1 Testing Phase Goal

The purpose of this document is to finalize the OnTask Platform and ensure that it meets the established objectives as well as that our clients have sufficient access to the means of using the OnTask Platform. During this phase, we will be focusing on testing each focus area of the OnTask Platform from the standpoint of extreme or unusual use regarding the financial and recruitment module components. In particular, we will test the components of financial and recruitment modules with respect to the following:

5.1.2 Test Strategy

All REST endpoints will be tested for proper functionality. The integrity of the React state will be verified by examining the ability of the React state to keep its values through multiple uses of the various modules, such as the User Security defined by Role, and lastly, Socket.io will be performance-tested for load.

5.2 Testing Tiers

1. The authentication controller (authController.js) and job controller (jobController.js) all functions will be run against the test environments.
2. Integration will be tested from React Context to Node.js API.
3. System will start testing from the beginning of positions generally until the time when the commission is released, all functional user journeys for each position will be tested from the beginning of hiring to when the end of the user originally received recognition as a result of that particular user journey.
4. The security audit will verify the authenticity of the JWTs and integrity of the Bcrypt hash.

5.3 Test Case Tables

Table 5.3.1: Authentication and Authorization Test Cases

ID	Scenario	Expected Result	Status
TC_AUT_1	Signup with existing email	Return 400 'User already exists'	Passed
TC_AUT_2	Login with wrong	Return 401 'Invalid credentials'	Passed

	password		
TC_AUT_3	Access /api/auth/profile without token	Return 401 'Not authorized'	Passed
TC_AUT_4	JWT Token Expiration	Redirect to Login after token expiry	Passed
TC_AUT_5	Bcrypt hashing verification	Database stores non-human readable string	Passed

Table 5.3.2: Job and Recruitment Module Test Case

ID	Scenario	Expected Result	Status
TC_JOB_1	Create job with missing fields	Validation error returned from Mongoose	Passed
TC_JOB_2	Apply to same job twice	Return 400 'Already applied'	Passed
TC_JOB_3	Tasker hiring worker	Status updates to 'Hired' in the sub-document	Passed
TC_JOB_4	Milestone progress > 100	Frontend validation block	Passed
TC_JOB_5	Screening question empty response	Requirement check on submission	Passed

Table 5.3.3: Escrow and Financial Security Test Case

ID	Scenario	Expected Result	Status
TC_FIN_1	Razorpay Payment verification failure	Escrow amount remains zero	Passed
TC_FIN_2	Release more than escrow amount	Backend logic block (400)	Passed
TC_FIN_3	Double withdrawal request	Only first request processed, status 'Pending'	Passed
TC_FIN_4	Total budget calculation	Escrow sum matches deposit success logs	Passed
TC_FIN_5	Bank detail validation (IFSC)	Regex check before profile save	Passed

5.4 Performance Benchmarks

Performance evaluation was conducted to ensure that the platform remains stable under load. The results are summarized in Table 6.4.

Table 5.4.1: Performance Evaluation

Operation	Sample Size	Avg Latency (ms)
Hiring API Request	100 calls	45ms
Escrow Release Loop	50 calls	120ms (with DB IO)
Socket Message Delivery	500 messages	18ms
Chart Data Processing	Profile view	22ms

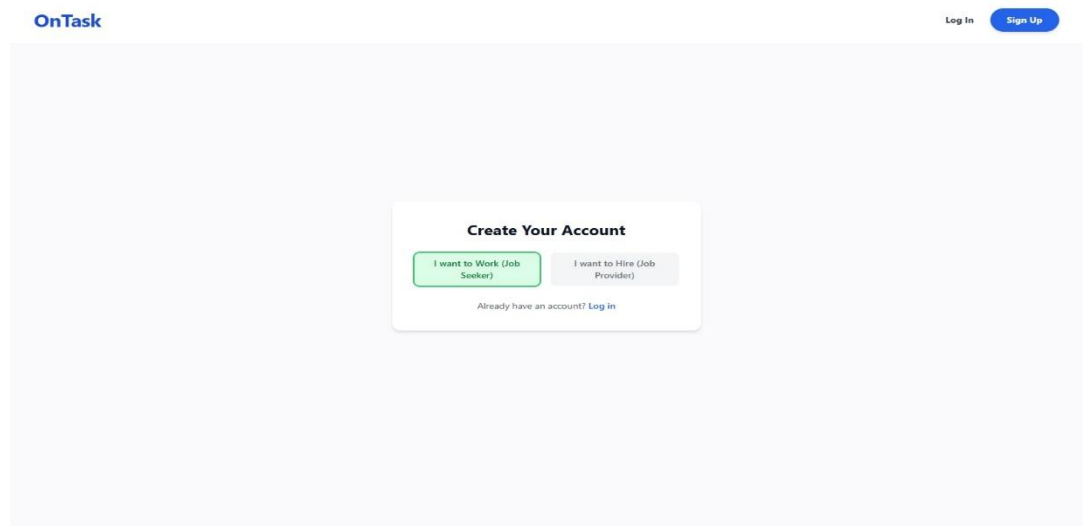
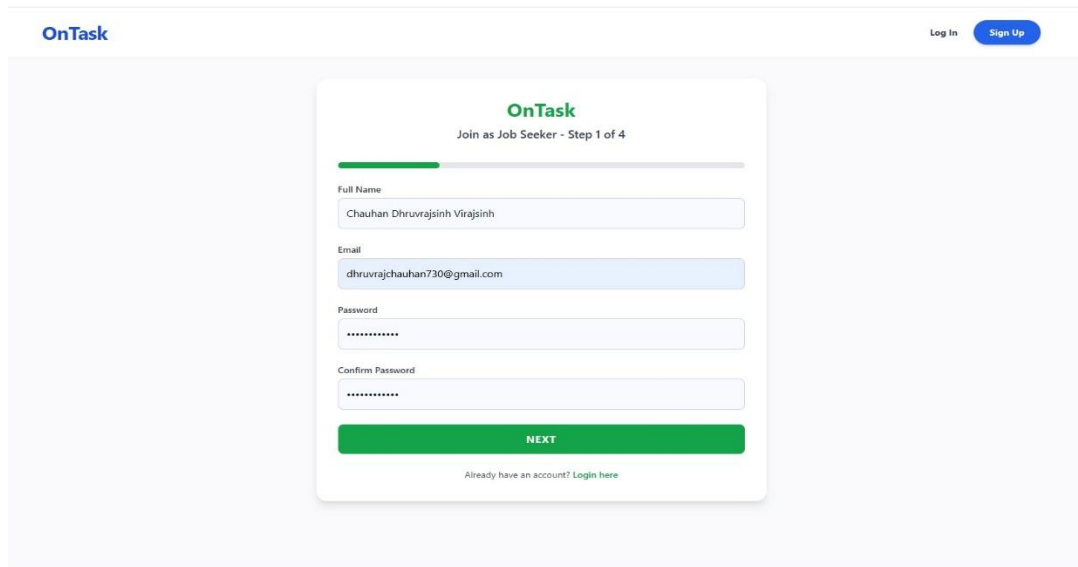


Figure 5.4.1: Signup Page



The image shows the first step of the OnTask job seeker signup process. The page has a header with the OnTask logo on the left and 'Log In' and 'Sign Up' buttons on the right. The main content area features a white card with the OnTask logo and the text 'Join as Job Seeker - Step 1 of 4'. Below this is a progress bar. The form contains four input fields: 'Full Name' with the value 'Chauhan Dhruvrajsinh Virajsinh', 'Email' with 'dhruvrajchauhan730@gmail.com', 'Password' with masked characters, and 'Confirm Password' with masked characters. A green 'NEXT' button is at the bottom of the form, and a link 'Already have an account? Login here' is below it.

OnTask

Log In Sign Up

OnTask

Join as Job Seeker - Step 1 of 4

Full Name

Chauhan Dhruvrajsinh Virajsinh

Email

dhruvrajchauhan730@gmail.com

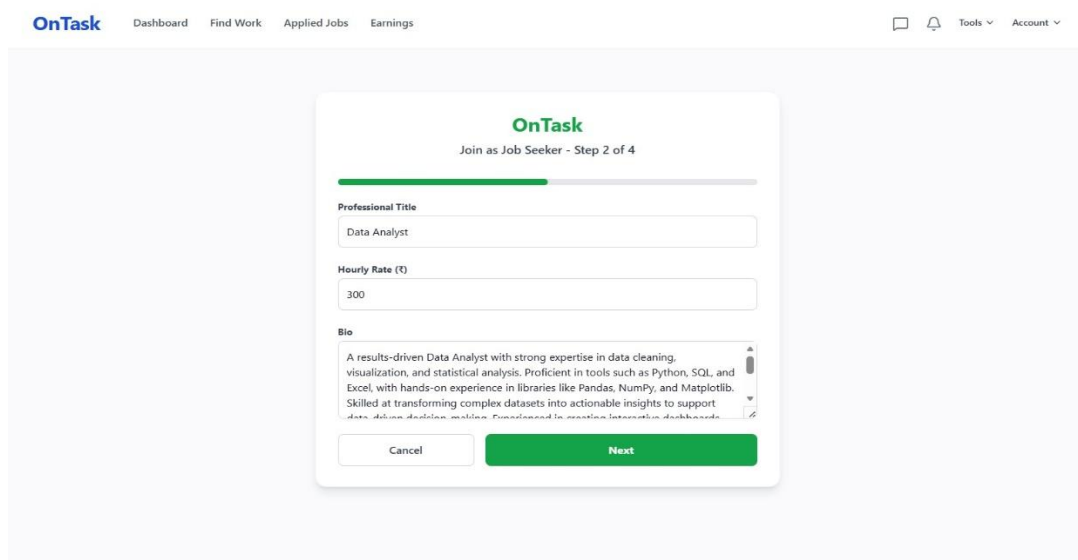
Password

Confirm Password

NEXT

Already have an account? [Login here](#)

Figure 5.4.2: Job Seeker Signup Step-1



The image shows the second step of the OnTask job seeker signup process. The page has a header with the OnTask logo on the left and navigation links 'Dashboard', 'Find Work', 'Applied Jobs', and 'Earnings' in the center. On the right are icons for chat, notifications, and a dropdown menu for 'Tools' and 'Account'. The main content area features a white card with the OnTask logo and the text 'Join as Job Seeker - Step 2 of 4'. Below this is a progress bar. The form contains three input fields: 'Professional Title' with the value 'Data Analyst', 'Hourly Rate (₹)' with '300', and a 'Bio' field with a text area containing a detailed description of the user's skills. At the bottom of the form are 'Cancel' and 'Next' buttons.

OnTask

Dashboard Find Work Applied Jobs Earnings

Tools Account

OnTask

Join as Job Seeker - Step 2 of 4

Professional Title

Data Analyst

Hourly Rate (₹)

300

Bio

A results-driven Data Analyst with strong expertise in data cleaning, visualization, and statistical analysis. Proficient in tools such as Python, SQL, and Excel, with hands-on experience in libraries like Pandas, NumPy, and Matplotlib. Skilled at transforming complex datasets into actionable insights to support data-driven decision-making. Experienced in creating interactive dashboards.

Cancel Next

Figure 5.4.3: Job Seeker Signup Step-2

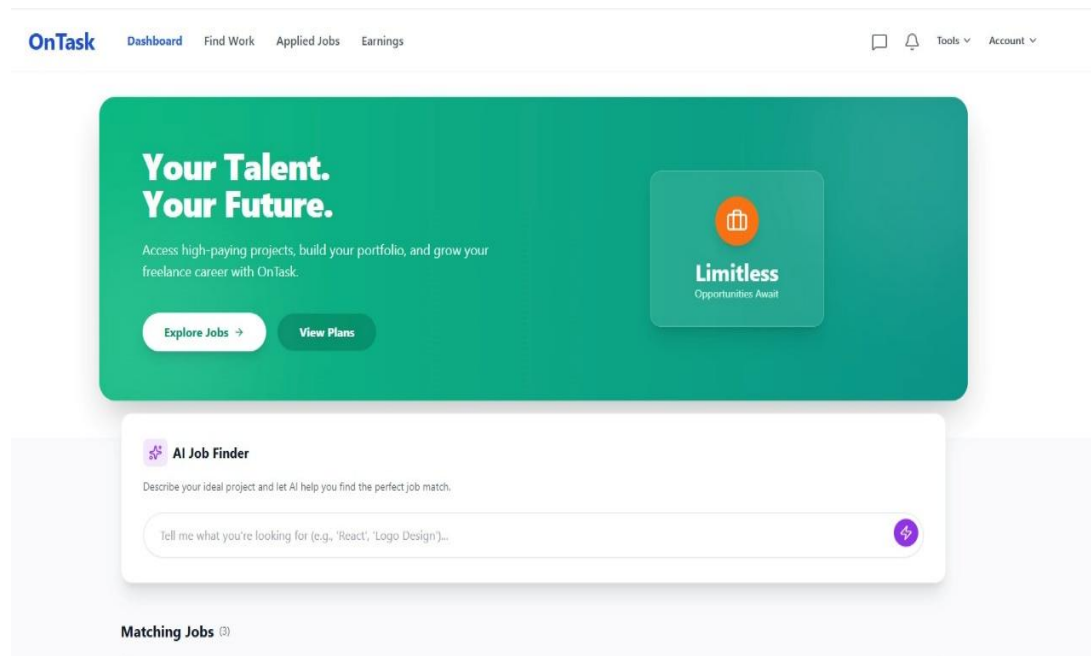


Figure 5.5.4: Job Seeker Home Page

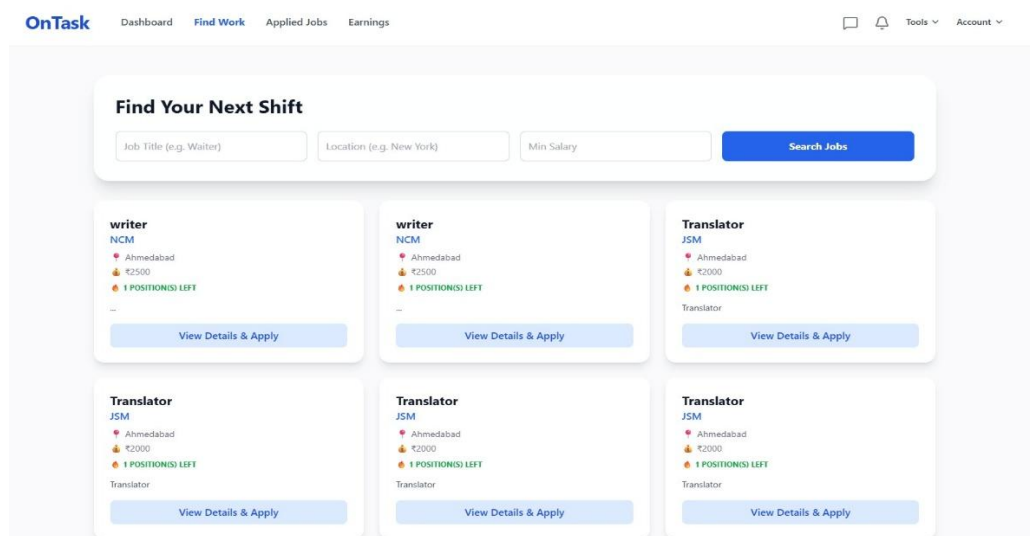


Figure 5.5.5: Job Seeker Finding Job

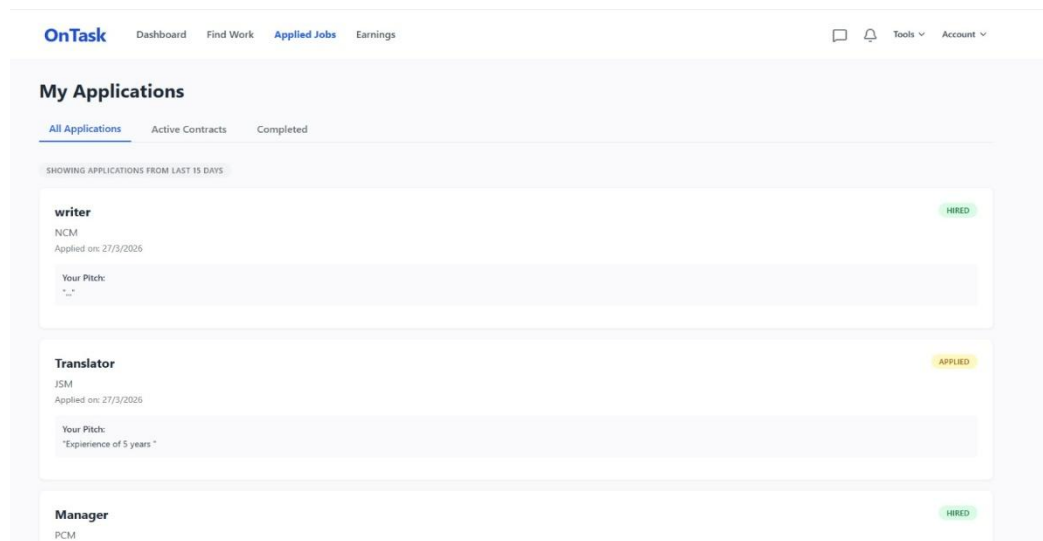


Figure 5.5.6: Job Seeker Applied Jobs

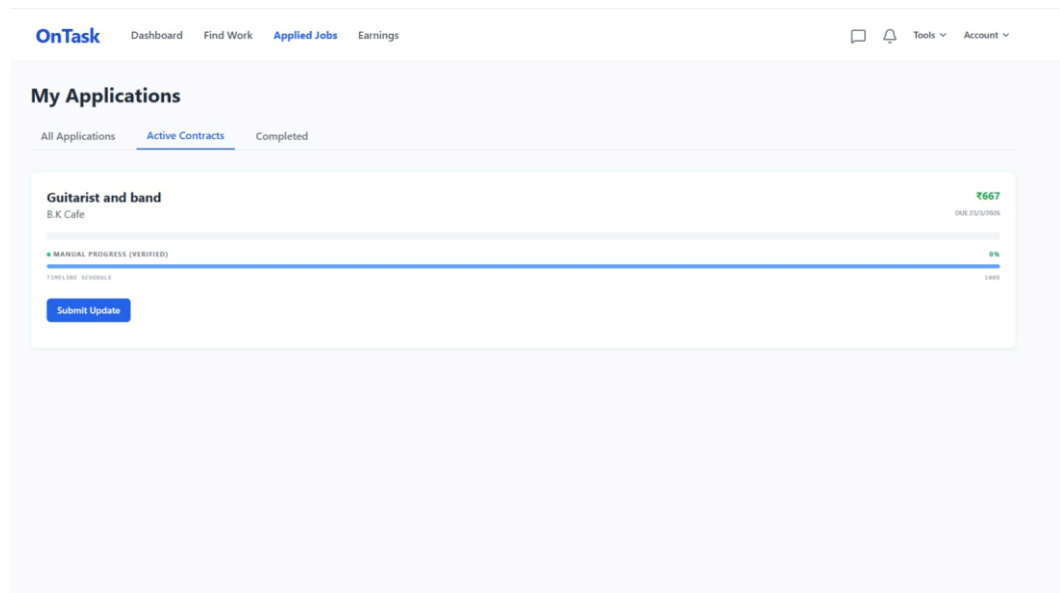


Figure 5.5.7: Job Seeker Active Contracts

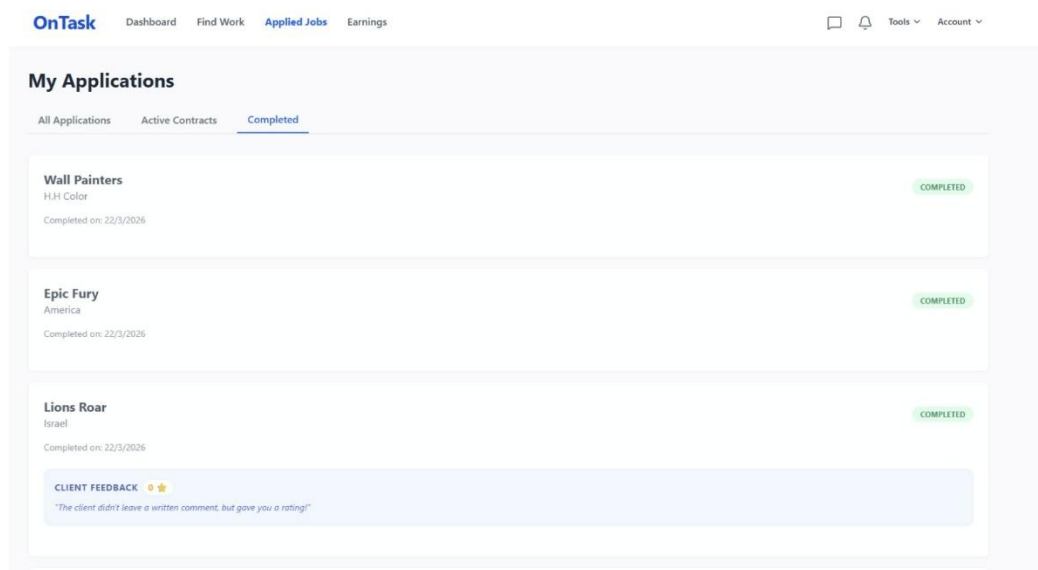


Figure 5.5.8: Job Seeker Completed jobs

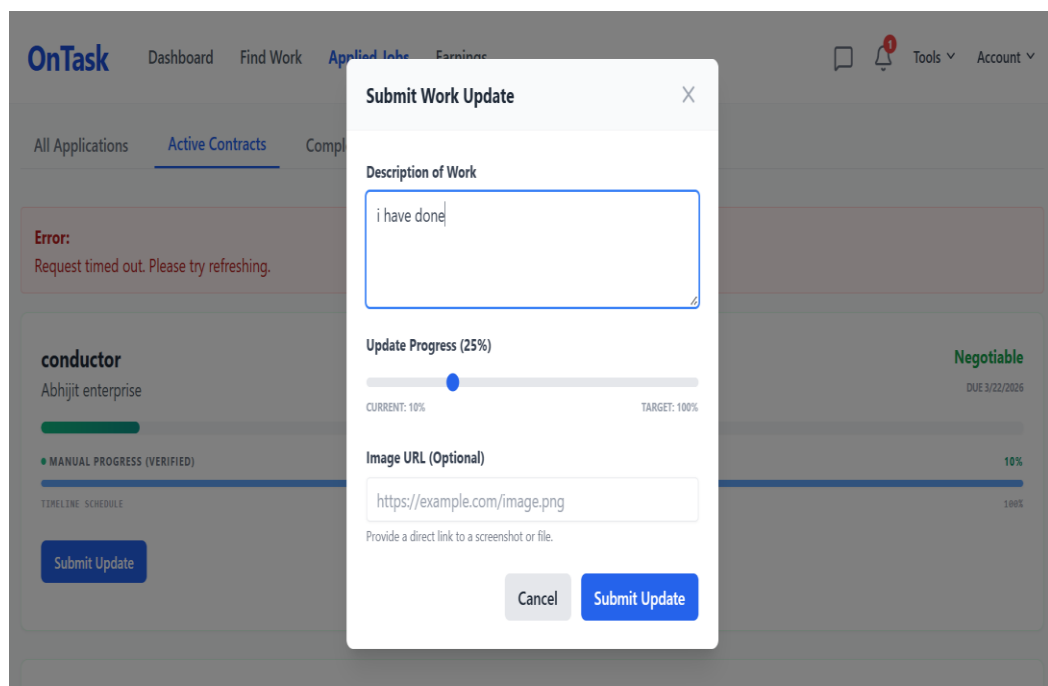


Figure 5.5.9: Job Seeker Submit Updates

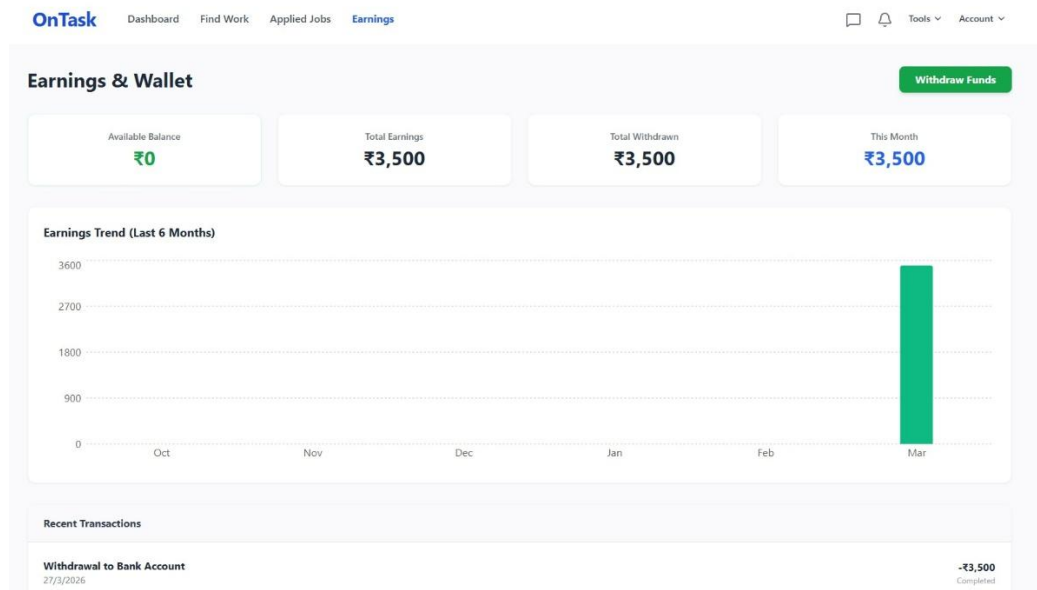


Figure 5.5.10: Job Seeker Earnings Window

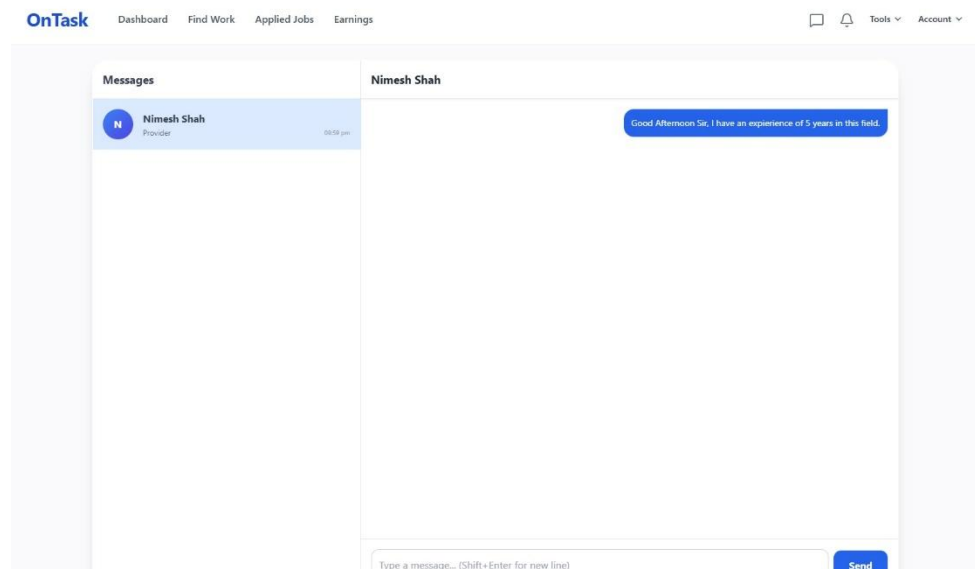


Figure 5.5.11: Job Seeker Message Section

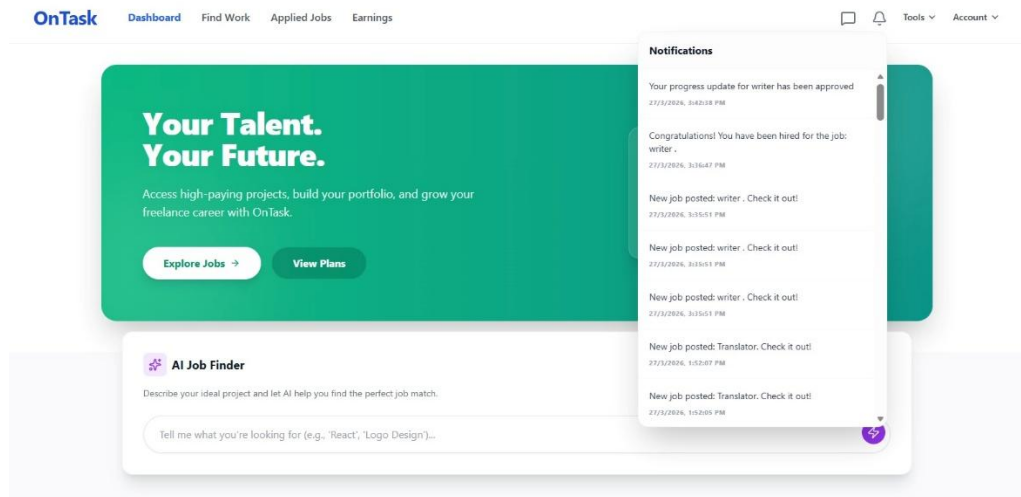


Figure 5.5.12: Job Seeker Notification Section

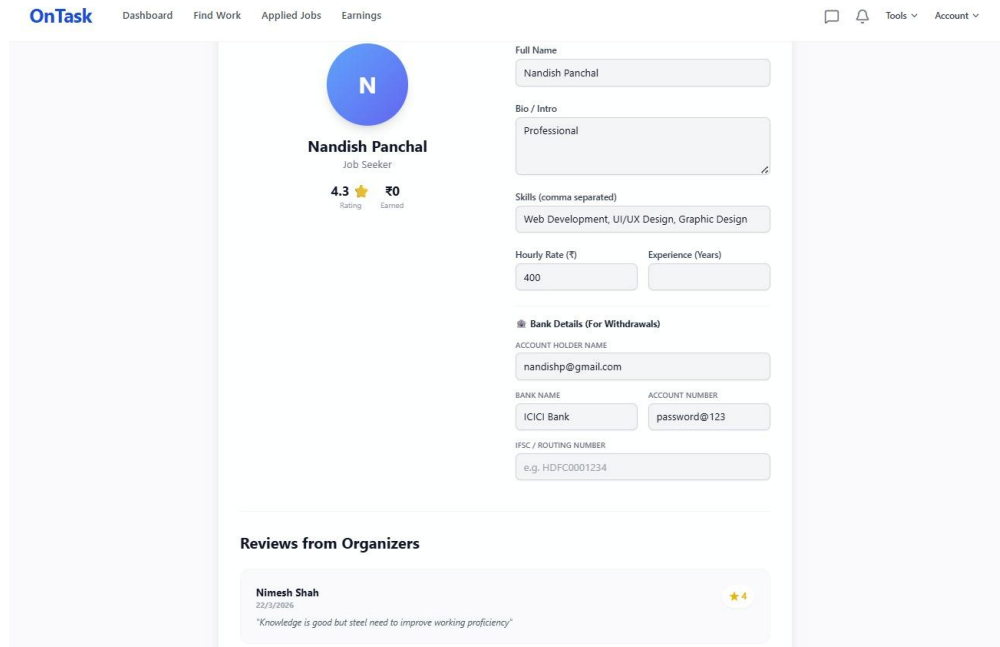


Figure 5.5.13: Job Seeker Profile Page

Figure 5.5.14: Job Seeker Profile Settings

Figure 5.5.15: Job Provider Signup Step-1

OnTask

Join as Provider- Step 2 of 4

Company / Organization Name

TOC

Industry

Event Management

Website (Optional)

https://...

Company Bio

A dynamic event management company specializing in planning and executing seamless, memorable events for corporate, social, and private clients. With expertise in event coordination, vendor management, budgeting, and creative design, the company delivers cost-effective solutions tailored to client needs.

Cancel Next

Figure 5.5.16: Job Provider Signup Step-2

OnTask

Join as Provider- Step 3 of 4

What talent are you looking for?

☐ Bartenders ☐ Servers

☒ DJs ☐ Security Personnel

☒ Event Planners ☐ Photographers

☒ Videographers ☒ Decorators

☒ Catering Staff ☐ Cleaning Crew

☐ Valet Drivers ☐ Promoters

Add Other Roles

Type a role and press Enter

Back Next

Figure 5.5.17: Job Provider Signup Step-3

OnTask
Dashboard
Post Job
My Listings
Find Talent

Tools
Account

OnTask

Join as Provider- Step 4 of 4

Verify Your Business

Provide your business address for verification and invoicing.

Street Address

Venus Parkland

City

Ahmedabad

State / Province

Gujarat

ZIP / User Code

380051

Tax ID (Optional)

XX-XXXXXXX

Skip for Now

Complete Profile

Figure 5.5.18: Job Provider Signup Step-4

OnTask
Dashboard
Post Job
My Listings
Find Talent

Tools
Account

Build Your Dream Team

Connect with top talent, manage projects effortlessly, and scale your business with OnTask.

Upgrade Plan

Browse Talent

A

S

M

Verified Professionals

Welcome Back

Manage your projects and find top talent

Available Freelancers

View All Talent

John Doe

Freelancer

★ 0 (0 yrs exp)

₹50/hr

Nandish Panchal

Designer

★ 4.3 (0 yrs exp)

₹400/hr

Dhruvrajsinh Chauhan

rrf

★ 0 (0 yrs exp)

₹184/hr

Dhruvrajsinh Chauhan

Freelancer

★ 0 (0 yrs exp)

₹0/hr

Figure 5.5.19: Job Provider Home Page

25

OnTask Dashboard **Post Job** My Listings Find Talent

Post a New Project
Find the perfect talent for your event or business.

[Manual Post](#) [AI Job Poster](#)

Job Title
Data Analyst

Company Name
NCM

Location
Ahmedabad

Salary Text
Negotiable

Numeric Budget (₹)
2500

Starting Date
28-03-2026

Work Duration
1 Days

Number of Freelancers Needed
1

Job Description
Detailed description of the role...

Figure 5.5.20: Job Provider Job Post

OnTask Dashboard Post Job **My Listings** Find Talent

My Jobs [Post New Job](#)

[All](#)
[Open](#)
[In Progress](#)
[Completed](#)

Data Analyst
Posted 28/3/2026

TOTAL BUDGET
₹2500

APPLICATIONS
0

STATUS
Open

HIRED (0)
Not yet hired

[Manage Job](#)

Applications (0)

Figure 5.5.21: Job Provider Posted Jobs

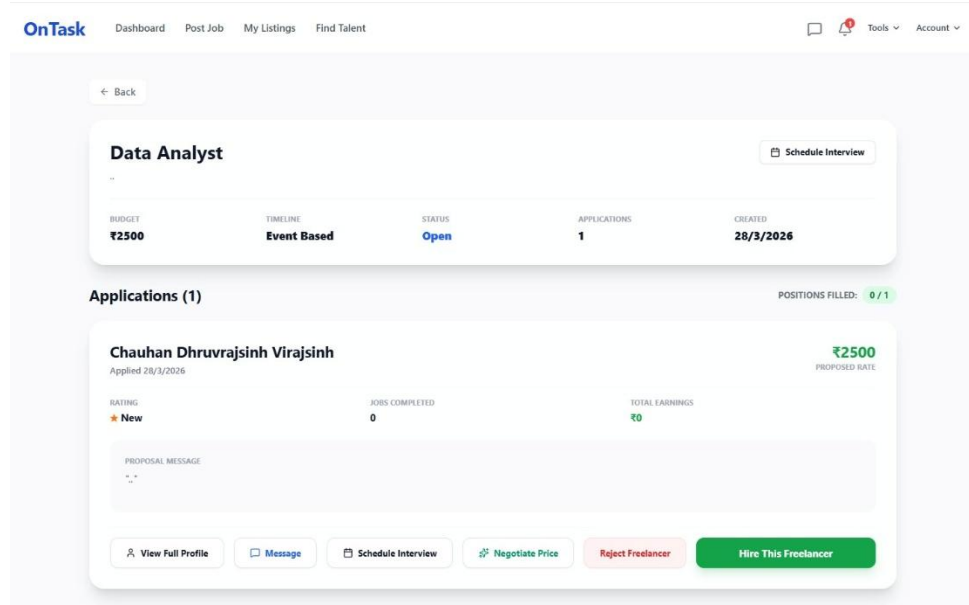


Figure 5.5.22: Applications of Job Seekers

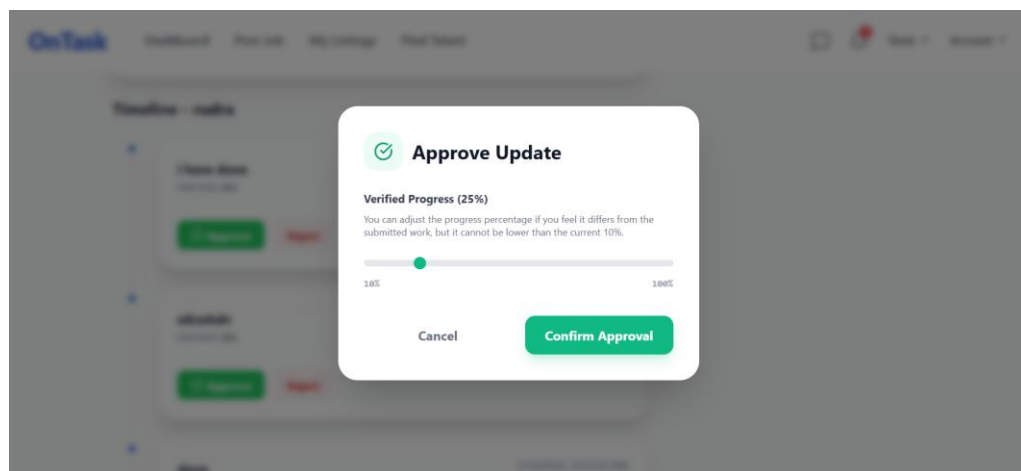


Figure 5.5.23: Job Provider Approve progress

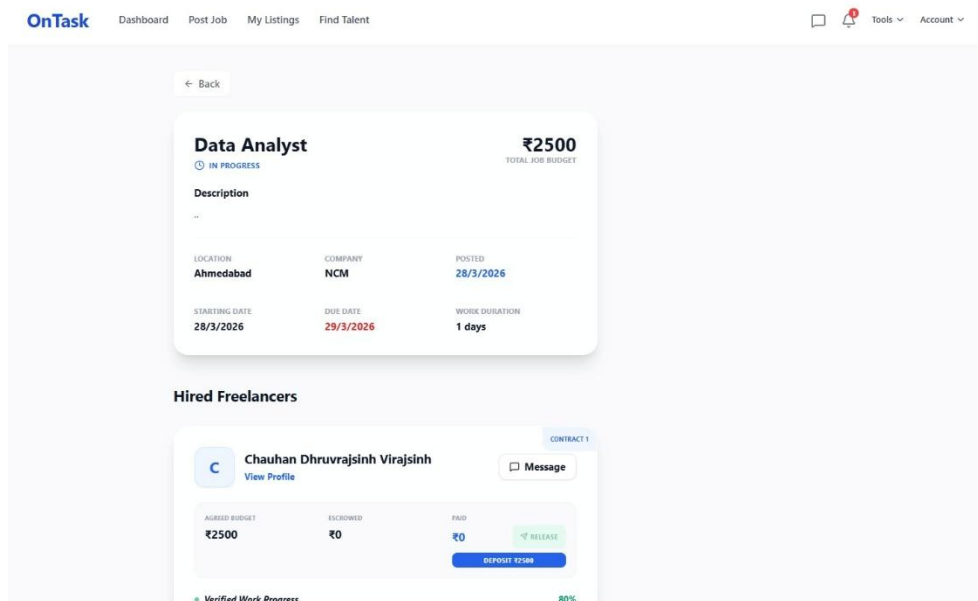


Figure 5.5.24: Job Provider Active Jobs

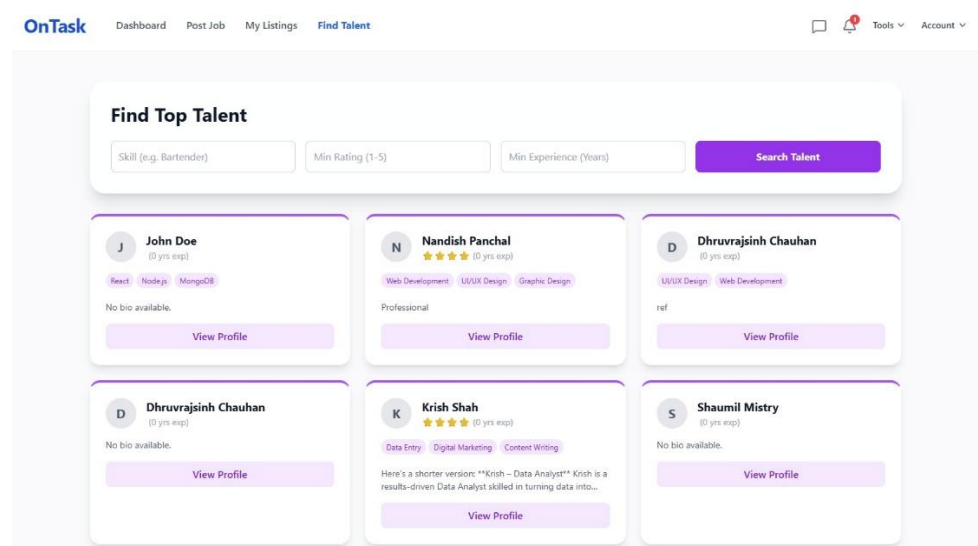


Figure 5.5.25: Job Provider Find Job Seekers

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

OnTask has addressed and successfully bridged the trust gap inherent in the Localised Gig Economy. The business model that incorporates a multi-role, end-to-end Marketplace for Recruiters to connect with Talent and assure Workers of payment completion was achieved by designing a resilient Escrow logic to handle the receipt and processing of asynchronous Razorpay webhooks while maintaining accurate state persistence. Overall, This project demonstrates that current MERN architectural designs provide the necessary platform for developing high integrity Financial Application solutions.

6.2 Limitations

- Milestones are released by the employer manually.
- Disputes must be mediated through intervention via the admin console.
- Payments can only be processed in INR via Razorpay.

6.3 Future Enhancements

6.3.1 Artificial Intelligence Empowered Resolutions of Disputes

To develop AI powered arbitration processes, we will create an AI application that can automatically resolve simple disputes for users by reviewing evidence of work (images of work completed) vs. project description and allow decisions to be rendered without involving humans as arbitrators.

6.3.2 Bluetooth Smart Contracts Created With Blockchain Technology

To develop AI powered arbitration processes, we will create an AI application that can automatically resolve simple disputes for users by reviewing evidence of work (images of work completed) vs. project description and allow decisions to be rendered without involving humans as arbitrators.

6.3.3 Mobile Apps Built On React Native

Because many semi-skilled workers tend to use their smart phones as their primary means of communication, we will develop an iOS/Android Native application using React Native to provide access through the same back-end API infrastructure.

6.3.4 Real-time Video Interviewing

Integrating WebRTC will enable recruiters to conduct video interviews online on OnTask, resulting in additional efficiencies for the hiring process.

APPENDICES

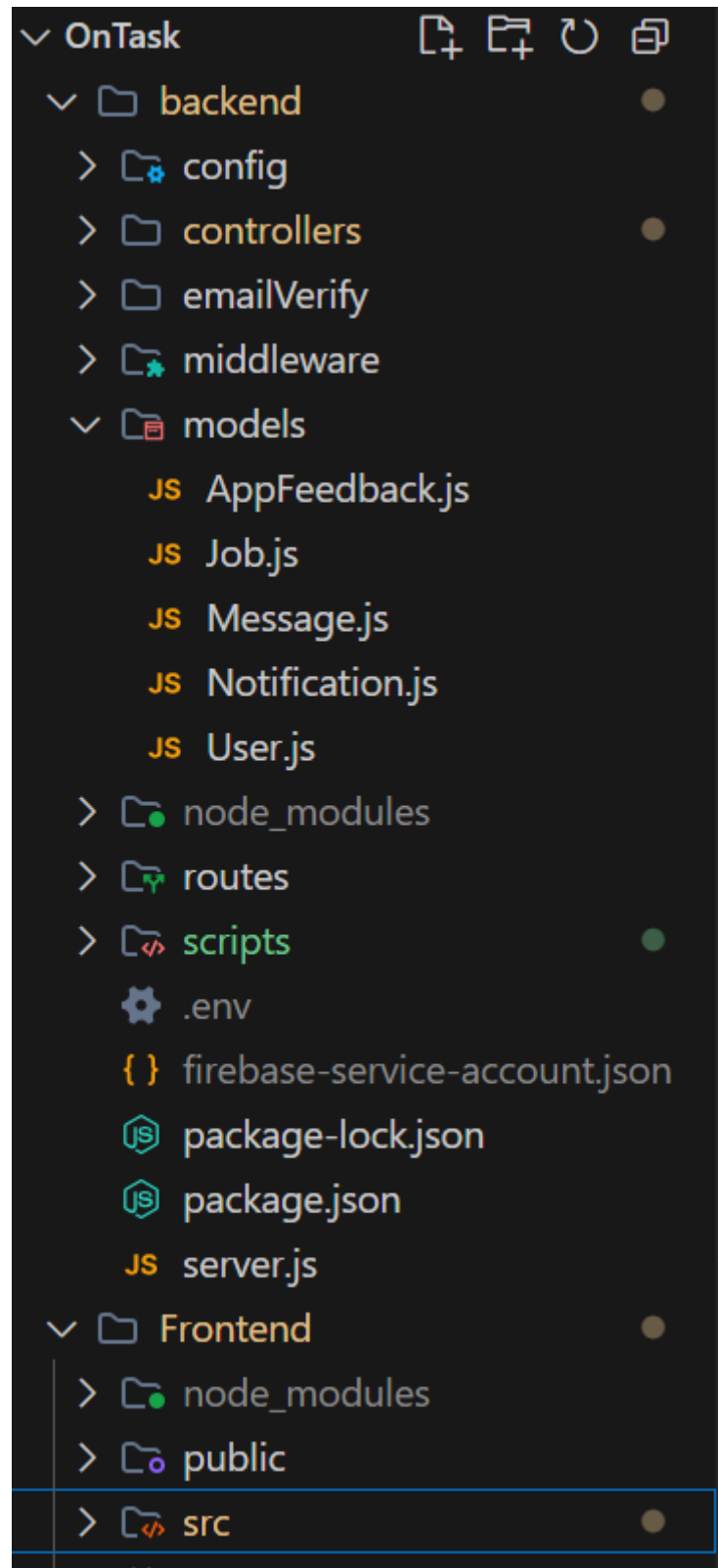


Figure 7.1.1: MVC Architecture

```

backend > config > JS pushNotificationService.js > ...
1  const admin = require('firebase-admin');
2  const path = require('path');
3
4  let serviceAccount;
5
6  try {
7    if (process.env.FIREBASE_SERVICE_ACCOUNT_PATH) {
8      serviceAccount = require(path.resolve(__dirname, '..',
process.env.FIREBASE_SERVICE_ACCOUNT_PATH));
9    } else {
10     serviceAccount = require('../firebase-service-account.json');
11    }
12
13    admin.initializeApp({
14      credential: admin.credential.cert(serviceAccount)
15    });
16    console.log('Firebase Admin initialized successfully');
17  } catch (error) {
18    console.error('Failed to initialize Firebase Admin. Push
notifications may not work:', error.message);
19  }
20
21  /**
22   * Sends a push notification to a specific device via FCM token
23   * @param {string} fcmToken - The FCM token of the recipient device
24   * @param {string} title - The notification title
25   * @param {string} body - The notification body
26   * @param {string} link - The URL to open when notification is clicked
27   */

```

Figure 7.1.2: Push Notification Config

```

backend > emailVerify > JS sendOtpMail.js > ...
1  const nodemailer = require('nodemailer');
2  require('dotenv').config();
3
4  const sendOtpMail = async (email, otp) => {
5    const transporter = nodemailer.createTransport({
6      host: 'smtp-relay.brevo.com',
7      port: 587,
8      auth: {
9        user: process.env.SMTP_USER,
10       pass: process.env.SMTP_PASS
11     }
12   });
13
14   const mailOptions = {
15     from: process.env.SMTP_EMAIL,
16     to: email,
17     subject: 'Password Reset OTP',
18     text: `Your OTP for password reset is: ${otp}. This OTP is
valid for 10 minutes.`
19   };
20
21   await transporter.sendMail(mailOptions);
22 };
23
24 module.exports = sendOtpMail;

```

Figure 7.1.3: Send OTP to Mail

```

backend > controllers > JS authController.js > ...
1  const jwt = require('jsonwebtoken');
2  const bcrypt = require('bcryptjs');
3  const User = require('../models/User');
4  const sendOtpMail = require('../emailVerify/sendOtpMail.js');
5
6  // JWT token generator
7  function generateToken(id) {
8      return jwt.sign({ id }, process.env.JWT_SECRET, {
9          expiresIn: '30d',
10     });
11 }
12
13 // @desc   Register new user
14 // @route  POST /api/auth/register
15 // @access Public
16 const registerUser = async (req, res) => {
17     try {
18         const { name, email, password, role, country, bankDetails,
19             companyName, industry, website, hiringNeeds, businessAddress, bio } =
20             req.body;
21         if (!name || !email || !password) {
22             return res.status(400).json({ message: 'Please add all
23             fields' });
24         }
25
26         // Check if user exists
27         const userExists = await User.findOne({ email });

```

Figure 7.1.4: Auth Controller

```

backend > controllers > JS paymentController.js > ...
1  const User = require('../models/User');
2  const Job = require('../models/Job');
3  const Razorpay = require('razorpay');
4  const crypto = require('crypto');
5
6  const razorpay = new Razorpay({
7      key_id: process.env.RAZORPAY_KEY_ID,
8      key_secret: process.env.RAZORPAY_KEY_SECRET,
9  });
10
11 // @desc   Subscribe to a plan (Mock)
12 // @route  POST /api/payment/subscribe
13 // @access Private
14 const subscribeUser = async (req, res) => {
15     try {
16         const { plan } = req.body; // 'starter', 'pro', 'elite'
17
18         const user = await User.findById(req.user.id);
19
20         if (user) {
21             // Ensure subscription object exists
22             if (!user.subscription) {
23                 user.subscription = { plan: 'none', status: 'none' };
24             }
25
26             user.subscription.plan = plan;
27             user.subscription.status = 'active';
28             // Set expiry to 30 days from now
29             const expiry = new Date();

```

Figure 7.1.5: Payment Controller

```

backend > controllers > JS applicationController.js > ...
1  const Job = require('../models/Job');
2
3  // @desc    Get my applications (Frealncr)
4  // @route    GET /api/applications/my-applications
5  // @access    Private
6  const getMyApplications = async (req, res) => {
7      try {
8          const jobs = await Job.find({
9              'applications.applicant': req.user.id
10         })
11         .populate('employer', 'name email');
12
13         const applications = jobs.reduce((acc, job) => {
14             if (!job.applications || !Array.isArray(job.applications))
15             return acc;
16
17             // Find the specific application for this user
18             const application = job.applications.find(
19                 app => app.applicant && app.applicant.toString() ===
20                 req.user.id
21             );
22
23             if (!application) return acc;
24
25             acc.push({
26                 jobId: job._id,
27                 jobTitle: job.title,
28                 company: job.company || 'Unknown Company',
29                 salary: job.salary || 'N/A',
30                 status: application.status || 'pending'
31             });
32         }, []);
33     } catch (error) {
34         res.status(500).json({ message: error.message });
35     }
36 }

```

Figure 7.1.6: Application Controller

```

backend > controllers > JS walletController.js > ...
1  const User = require('../models/User');
2
3  // @desc    Get wallet data
4  // @route    GET /api/wallet
5  // @access    Private
6  const getWalletData = async (req, res) => {
7      try {
8          const user = await
9          User.findById(req.user.id).select('walletBalance totalEarnings
10         totalWithdrawn transactions');
11
12         if (!user) {
13             return res.status(404).json({ message: 'User not found' });
14         }
15         res.json(user);
16     } catch (error) {
17         res.status(500).json({ message: error.message });
18     }
19 }
20
21 // @desc    Request withdrawal
22 // @route    POST /api/wallet/withdraw
23 // @access    Private
24 const withdrawFunds = async (req, res) => {
25     try {
26         const { amount } = req.body;
27         const withdrawAmount = Number(amount);
28
29         if (!withdrawAmount || withdrawAmount <= 0) {
30             return res.status(400).json({ message: 'Please enter a
31             valid amount' });
32         }
33     } catch (error) {
34         res.status(500).json({ message: error.message });
35     }
36 }

```

Figure 7.1.7: Wallet Controller

```

1  import React, { useState, useEffect, useContext } from 'react';
2  import axios from 'axios';
3  import AuthContext from '../context/AuthContext';
4  import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip,
   ResponsiveContainer } from 'recharts';
5
6  const EarningsPage = () => {
7      const { user, token } = useContext(AuthContext);
8      const [walletData, setWalletData] = useState(null);
9      const [loading, setLoading] = useState(true);
10     const [showWithdrawModal, setShowWithdrawModal] = useState(false);
11     const [withdrawAmount, setWithdrawAmount] = useState('');
12
13     useEffect(() => {
14         fetchWalletData();
15     }, []);
16
17     const fetchWalletData = async () => {
18         try {
19             const config = {
20                 headers: { Authorization: `Bearer ${token}` }
21             };
22             const res = await axios.get('/api/wallet', config);
23             setWalletData(res.data);
24             setLoading(false);
25         } catch (error) {
26             console.error('Error fetching wallet data:', error);
27             setLoading(false);
28         }
29     };

```

Figure 7.1.8: Earning React Page

```

Frontend > src > pages > MessagesPage.jsx > ...
1  import { useState, useEffect, useContext, useRef } from 'react';
2  import axios from 'axios';
3  import AuthContext from '../context/AuthContext';
4  import GlassContainer from '../components/premium/GlassContainer';
5  import { useLocation } from 'react-router-dom';
6
7  const MessagesPage = () => {
8      const { user, socket } = useContext(AuthContext);
9      const [conversations, setConversations] = useState([]);
10     const [currentChat, setCurrentChat] = useState(null);
11     const [messages, setMessages] = useState([]);
12     const [newMessage, setNewMessage] = useState('');
13     const messagesEndRef = useRef(null);
14
15     // Check for userId passed via state (from "Interview" button)
16     const location = useLocation();
17     const initialChatUserId = location.state?.recipientId ||
location.state?.userId;
18
19     useEffect(() => {
20         if (user) {
21             fetchConversations();
22         }
23     }, [user]);
24
25     useEffect(() => {
26         if (initialChatUserId && conversations.length > 0) {
27             const existingConv = conversations.find(c => c._id ===
initialChatUserId);
28             if (existingConv) {

```

Figure 7.1.9: Message React Page

```

Frontend > src > pages > ResetPasswordPage.jsx > ...
5  const ResetPasswordPage = () => {
6      const [formData, setFormData] = useState({
7          newPassword: '',
8          confirmPassword: ''
9      });
10     const { newPassword, confirmPassword } = formData;
11     const [message, setMessage] = useState('');
12     const [error, setError] = useState('');
13     const [showNewPassword, setShowNewPassword] = useState(false);
14     const [showConfirmPassword, setShowConfirmPassword] =
15         useState(false);
16     const navigate = useNavigate();
17     const [searchParams] = useSearchParams();
18     const email = searchParams.get('email');
19
20     useEffect(() => {
21         if (!email) {
22             navigate('/forgot-password');
23         }
24     }, [email, navigate]);
25
26     const onChange = (e) => {
27         setFormData((prevState) => ({
28             ...prevState,
29             [e.target.name]: e.target.value
30         }));
31     };
32
33     const onSubmit = async (e) => {

```

Figure 7.1.10: Reset Password React Page

```

Frontend > src > components > FeedbackModal.jsx > ...
6
7  const FeedbackModal = ({ isOpen, onClose, freelancerId, jobId,
8      freelancerName, onFeedbackSubmitted }) => {
9      const [freelancerRating, setFreelancerRating] = useState(0);
10     const [freelancerHoverRating, setFreelancerHoverRating] =
11         useState(0);
12     const [freelancerComment, setFreelancerComment] = useState('');
13
14     const [appRating, setAppRating] = useState(0);
15     const [appHoverRating, setAppHoverRating] = useState(0);
16     const [appFeedback, setAppFeedback] = useState('');
17
18     const [isSubmitting, setIsSubmitting] = useState(false);
19
20     if (!isOpen) return null;
21
22     const handleSubmit = async (e) => {
23         e.preventDefault();
24
25         if (freelancerRating === 0) {
26             toast.error('Please provide a rating for the freelancer.');
```

Figure 7.1.11: Feedback Model React UI

```

Frontend > src > components > AIChatbot.jsx > ...
4
5  const AIChatbot = () => {
6      const [isOpen, setIsOpen] = useState(false);
7      const [messages, setMessages] = useState([
8          { id: 1, text: "Hi! I'm OnTask AI. I can help you find jobs,
          find talent, or post a new job instantly. Try typing 'I need a
          bartender' or 'Find driver jobs'.", sender: 'bot' }
9      ]);
10     const [input, setInput] = useState('');
11     const navigate = useNavigate();
12     const messagesEndRef = useRef(null);
13
14     const scrollToBottom = () => {
15         messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
16     };
17
18     useEffect(() => {
19         scrollToBottom();
20     }, [messages]);
21
22     const handleSend = () => {
23         if (!input.trim()) return;
24
25         const userMsg = { id: Date.now(), text: input, sender: 'user'
26     };
27
28     setMessages(prev => [...prev, userMsg]);
29     setInput('');
30
31     // Simulate AI "Thinking"
32     setTimeout(() => {

```

Figure 7.1.12: AI Chatbot React UI

```

7
8  const TaskerSearch = () => {
9      const { user } = useContext(AuthContext);
10     const [jobs, setJobs] = useState([]);
11     const [filters, setFilters] = useState({
12         title: '',
13         location: '',
14         minSalary: ''
15     });
16
17     const searchJobs = async () => {
18         try {
19             const params = new URLSearchParams(filters);
20             const res = await axios.get(`/api/search/jobs?${params}`);
21             setJobs(res.data);
22         } catch (error) {
23             console.error(error);
24         }
25     };
26
27     useEffect(() => {
28         searchJobs();
29     }, []);
30
31     const handleChange = (e) => {
32         setFilters({ ...filters, [e.target.name]: e.target.value });
33     };
34
35

```

Figure 7.1.13: Tasker Search React Page

REFERENCES

- [1] L. Guseek and M. Wiesche, "IT Professionals in the Eig economy," in *proc. int. conf. information systems*, 2023.
- [2] L. Gussek, M. Wiesche, and H. Krcmar, "Challenges of IT Freelancers on Digital Labor Platforms," *springer*, 2023.
- [3] V. Raghupathi and W. Raghupathi, "Exploring Influential Factors in Hiring Freelancers in Online Labor Platforms," *j. information systems*, 2023.
- [4] A. Pinna and S. Ibba, "A Blockchain-Based Decentralized System for Temporary Employment Contracts," *ieee*, 2017.
- [5] D. Vakharia and M. Lease, "Beyond Amazon Mechanical Turk: Crowd Work Platforms," in *proc. ieee conf.*, 2013.
- [6] F. Ricci, L. Rokach, and B. Shapira, *recommender systems handbook*, *springer*, 2015.
- [7] J. Brown, "Real-Time Communication in Web Applications," *ieee communications*, 2019.
- [8] K. Verma and R. Singh, "Secure Payment Gateway Integration in Web Applications," *int. j. computer science*, 2020.
- [9] S. Patel and R. Mehta, "Design and Implementation of Web-Based Job Portal System," *int. j. engineering research and technology (ijert)*, 2021.
- [10] L. Chen, "Digital Workforce Platforms and Gig Economy," *ieee access*, 2021.
- [11] J. Horton and L. Chilton, "The Labor Economics of Paid Crowdsourcing," in *proc. acm conf. electronic commerce*, 2010.
- [12] A. Kittur et al., "The Future of Crowd Work," in *proc. acm conf. computer supported cooperative work*, 2013.
- [13] V. De Stefano, "The Rise of the Just-In-Time Workforce: On Demand Work, Crowd Work and Labour Protection," *int. labour office*, 2016.
- [14] A. Sundararajan, *The Sharing Economy: The End of Employment and The Rise of Crowd-Based Capitalism*, *mit press*, 2016.
- [15] F. Ricci, L. Rokach, and B. Shapira, "Introduction to Recommender Systems," *springer*, 2015.